THE FOUR PROGRAMS IN THIS REPO

Each program does one thing well. Pick the right tool for what you are doing.

Program	What it does	When to use it	Failsafes?
can_sniffer.py	Passive listener. Auto-detects baud, highlights known Tesla IDs, decodes 0x370 in real time.	FIRST every session. Before sending anything, confirm the bus is alive and you are on chassis CAN.	N/A (read-only)
move.py 5	CLI one-shot. Sends 0x488 with target = N degrees at 50 Hz with LPF smoothing.	Quick sanity check. "Does the wheel move?" Hard angle clamp +/- 90, no GUI.	Ctrl-C only
steer.py	Live keyboard control. GUI shows a steering wheel that rotates with commanded angle.	Drive the wheel like a video game. Test sign convention. Do demos.	ESC, Q, window close, hard +/- 60 clamp
tesla_steering_test.py	Full-featured GUI. Slider, status panel, event log, EAC transition logging, divergence trip, watchdog timeout.	Real testing. Anything safety-critical. The version Charlie used to validate the protocol.	All of them: ESC, big red E-STOP, RX timeout, divergence, bus error count, hard clamp, rate clamp

RECOMMENDED ORDER, EVERY SESSION

1. Run **can_sniffer.py** first. Confirm "TESLA CHASSIS CAN CONFIRMED" and turn the wheel by hand to verify the angle decode tracks. This proves the bus is alive AND that 0x370 is being read correctly.
2. Run **move.py 0**. Just engage at center. Wheel should not move. eacStatus should transition to ACTIVE. If it does, the protocol is healthy.
3. Run **move.py 5**, then **move.py -5**, then **move.py 0**. Confirm sign convention matches what Jordan saw on 03 May (positive = right).
4. Use **steer.py** for live control or **tesla_steering_test.py** for slider control.
5. If you see flicker, glitching, or HIGH_ANGLE_RATE_REQ errors, switch to **FLICKER_TROUBLESHOOTING.pdf**.

PRE-FLIGHT (CONFIRMED BEFORE EVERY SESSION)

Check	Why
Front wheels OFF the ground (jack stands or tie rods disconnected)	Standstill steering on loaded wheels stresses the rack and gives unreliable behavior.
Tesla in Drive Ready (key in, brake pressed, dash lit)	Rack needs full power, not Accessory.
12V battery healthy (> 12.4 V at rest)	Brown-outs cause weird rack behavior.
SYS TEC adapter: USB to laptop, DB9 to OBD-II	See WIRING_DIAGRAM.pdf for pin-by-pin.
Driver hands OFF the steering wheel during engage	Any applied torque causes HANDS_ON fault.

STEER.PY -- LIVE KEYBOARD STEERING

steer.py is a small dedicated GUI for driving the wheel with the arrow keys. Shows a steering-wheel icon that rotates in real time, plus commanded/measured/eacStatus/error readouts.

Install

If you already ran tesla_steering_test.py and it worked, no new install needed. python-can covers everything. steer.py uses Tkinter (built into Python) for the GUI, no extra packages.

```
python -m pip install python-can
```

Run

From the folder containing the .py files:

```
python steer.py
```

A 520x620 dark window opens. CAN bus opens automatically. After a few seconds the badge in the top-right reads ACTIVE (green) once the rack accepts our 0x488. **Click in the window** to give it focus, then use arrow keys.

Controls

Key	Action
LEFT arrow	Hold to decrease target angle (steer left). Release to hold position.
RIGHT arrow	Hold to increase target angle (steer right). Release to hold position.
SPACE	Snap target back to 0 (recenter the wheel).
Q	Disengage cleanly and exit.
ESC	Same as Q.
Close window	Same as Q.

GUI Tour

Element	What it shows
Top: "LIVE KEYBOARD STEERING"	Title.
Top right: status badge	Color-coded EAC state: green ACTIVE, yellow AVAILABLE, grey INHIBITED, red E-STOP / FAULT.
Center: steering-wheel icon	Three-spoke wheel that ROTATES with the commanded angle. Blue dot indicates the top of the wheel. Center hub shows the angle as a number.
Below the wheel: direction arrow	Yellow "< < < STEERING LEFT" or "STEERING RIGHT > > >" while you hold an arrow. "HOLDING" otherwise.
COMMANDED card	The LPF-smoothed angle we are currently sending in 0x488. Big number.
MEASURED card	The rack's reported actual wheel angle from 0x370.
EAC card	INHIBITED / AVAILABLE / ACTIVE / FAULT / SNA. Color-coded.
ERROR card	eacErrorCode name (NONE, MIN_SPEED, HIGH_ANGLE_RATE_REQ, etc.). Red if non-zero.
Bottom: log line	Single-line status. CAN open / E-STOP / TX errors.

Safety in steer.py

Three independent disengage paths. Any of them stops 0x488 and falls back to manual steering within ~200 ms (the rack's own timeout):

- **Q or ESC**: software disengage. Sends 5 frames with controlType=0 then exits cleanly.
- **Close the window**: same as Q.
- **Hard angle clamp** at +/- 60 deg. Holding an arrow forever will not push past the clamp.
- **Smoothing**: every commanded angle goes through the same LPF as move.py with tau=0.20s. Even slamming the arrow keys produces a smooth rate response, not step changes.

NOT in steer.py: RX timeout watchdog, divergence trip, bus error counter. If you need those, use tesla_steering_test.py instead. steer.py is the lighter tool for situations where you have eyes on the wheel and want quick keyboard control.

A/B TESTING THE THEORY BRANCHES

If you are seeing flicker (per FLICKER_TROUBLESHOOTING.pdf), the recommended next step is to A/B test the two theory branches I committed on 04 May. They are different ideas about what causes the rack's HIGH_ANGLE_RATE_REQ to fire.

Theory A vs Theory B at a glance

Aspect	theory-A-conservative-rate	theory-B-pure-filter
Hypothesis	Rate too aggressive at standstill	Rate-limit creates velocity discontinuity that the rack's differentiator reads as a spike
Code change	MAX_RATE: 50 -> 20 deg/s. tau: 0.15 -> 0.30 s	Rate limiter removed entirely. tau: 0.15 -> 0.40 s
Wheel motion feel	Slow constant ramp	Organic ease-in / ease-out
Big-angle command time (0 -> 30)	~1.5 sec	~1.0 sec but with tapered ends
Visual signature when working	Wheel tracks slider but slowly	Wheel motion looks human, settles smoothly
Best when error is...	HIGH_ANGLE_RATE_REQ during ramps	HIGH_ANGLE_RATE_REQ AND/OR jerky stops at end of ramps

Test procedure (do this for each branch)

Step 1 -- switch branch:

```
git fetch origin
git checkout theory-A-conservative-rate
```

Step 2 -- run the GUI:

```
python tesla_steering_test.py
```

Step 3 -- run a fixed sequence and write down the results:
- Connect, ENGAGE at 0 -- watch event log for transitions
- Slide to +5, hold 3 sec, slide back to 0
- Slide to +30, hold 3 sec, slide back to 0
- Slide to -30, hold 3 sec, slide back to 0
- DISENGAGE, DISCONNECT

Score each branch

Question	How to score
How often did EAC flicker?	Steady ACTIVE / occasional flicker / constant flicker
What error codes appeared?	List from the event log: e.g. HIGH_ANGLE_RATE_REQ x4, NONE rest
Wheel motion feel	Smooth ramps / smooth with kinks / jerky throughout
Big angle (30) reached without fault?	Yes / No / Partial (got stuck at X deg)
Sign convention still right?	Positive = right (sanity check)

Reporting back

Send Derek for each branch:
1. The 5 lines of scoring above
2. A screenshot of the GUI mid-test
3. The contents of the event log (right-click in the log area, paste into a text)

Then return to main:

```
git checkout main
```

RECOMMENDED FULL WORKFLOW

Once we know which theory branch wins (or both fail and we cherry-pick a hybrid), this is the production-style workflow for any future session:

Phase	Tool	What to verify
0. Pre-flight	Visual + multimeter	Wheels off ground, OBD-II pin 1-9 ~60 ohm, 12V healthy, ignition Drive Ready
1. Bus health	can_sniffer.py	"TESLA CHASSIS CAN CONFIRMED". Turn wheel by hand, decoded angle tracks.
2. Engage smoke test	move.py 0	EAC transitions to ACTIVE. No motion. Ctrl-C cleanly disengages.
3. Sign + small angles	move.py 5, -5, 10, -10	Wheel turns the expected direction. No HIGH_ANGLE_RATE_REQ in event log (run sniffer in parallel).
4. Live keyboard	steer.py	Arrow keys steer the wheel smoothly. Stay below +/- 60. Wheel icon rotates correctly.
5. Full GUI	tesla_steering_test.py	Full slider range, all failsafes verified (ESC, divergence trip, RX timeout). E-STOP test.
6. Documentation	Notes + screenshots	Anything unexpected goes in a session note in the field_testing/ folder of the repo.

VALIDATION CHECKLIST AFTER A FLICKER FIX LANDS

After applying any fix from FLICKER_TROUBLESHOOTING.pdf or a theory branch, run this checklist before declaring the issue resolved. All must pass:

Check	Pass criterion
Engage at 0 deg	EAC reaches ACTIVE within 2 seconds, holds steady for 10 seconds
Slide to +5 and back	No HIGH_ANGLE_RATE_REQ in event log. EAC stays ACTIVE the whole time.
Slide to +30 and back	Same. Wheel motion is visually smooth.
Slide to -30 and back	Same. Sign convention still positive=right.
E-STOP via ESC mid-motion	Wheel stops within ~200 ms. Status reads E-STOP.
Reconnect after E-STOP	DISCONNECT then CONNECT works. ENGAGE works without restart.
Bus Errors counter	Stays at 0 throughout the entire session.
Divergence (commanded vs measured)	< 2 deg in steady state. < 5 deg during ramps.
0x370 RX rate	> 80 frames/sec the whole time.

WHAT DOCS COVER WHAT

Document	Use it for
PINOUT_VERIFICATION.pdf	First-time bench setup. Confirming OBD-II pins 1/9 are populated and the bus is alive.
WIRING_DIAGRAM.pdf	Pin-by-pin wiring map between SYS TEC DB9 and OBD-II. Print and keep next to the laptop.
INSTRUCTION_MANUAL.pdf	First-time Windows install of Python, python-can, and the SYS TEC driver.
FLASH_AND_TROUBLESHOOTING.pdf	Re-flashing the EPAS firmware via the comma 3X. Use only if rack stays at INHIBITED.
FLICKER_TROUBLESHOOTING.pdf	Diagnosing and fixing EAC flicker / HIGH_ANGLE_RATE_REQ / wheel twitching.
IMPLEMENTATION_GUIDE.pdf (this doc)	Which program to use when. steer.py controls and GUI tour. Theory branch A/B testing.
TEST_SCRIPT.txt	Two-person live narration walkthrough of a full bench test session.
READ_ME_FIRST.txt	Where Jordan should start every session. Lists everything in order.